RTL NETLIST → LIBRARIES → ELABORATION → SDC & CHECKS → MAPPING → OPTIMIZATION → SIGNOFF

PD SYN

PD SYNTHESIS

PD Synthesis transforms the design intent
— into an optimized, manufacturable and signoff-ready layout. —

From RTL to GDSII — Engineering the Silicon Reality.

PD SYNTHESIS FLOW

1. FLOORPLAN	2. PLACEMENT	3. CTS	4. ROUTING	5. SIGNOFF
• Die & Core Area • IO Placement • Power Planning	• Cell Placement • Optimization • Timing Driven	• Clock Tree Synthesis • Skew Optimization	• Global Routing • Detailed Routing • DRC Clean	• Timing Signoff • DRC / LVS • QoR Signoff

PD SYNTHESIS OPTIMIZATION GOALS

Timing Closure | Area Minimization | Power Efficiency | Congestion Reduction | Signal Integrity | DFM Compliance

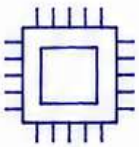
KEY ENGINES

TIMING ENGINE	CONGESTION ENGINE	POWER ENGINE	DRC / LVS ENGINE	QOR ENGINE
• STA • Slack Analysis • Path Optimization	• Hotspot Detection • Resource Balance • Re-Routing	• Power Estimation • IR Drop Analysis • Power Optimization	• Rule Checking • LVS Verification • Clean Signoff	• Area / Power / Timing • Multi-Corner Analysis • QoR Reporting

Good Synthesis Builds a Better Layout.
Better Layout Delivers a Better Silicon.

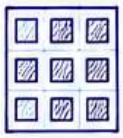
Author :
Konda Tharun Vivekananda

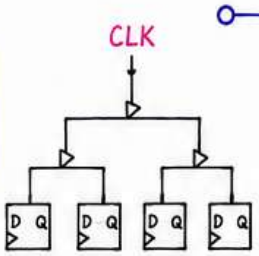
FLOORPLAN → PLACEMENT → CTS → ROUTING → SIGNOFF → GDSII





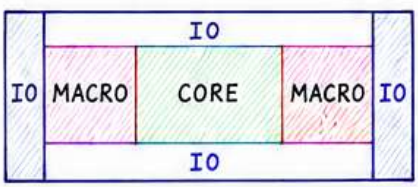
PD SYNTHESIS



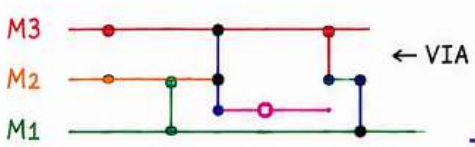


INDEX

S.No.	Main Heading / Concept	Page No.
1	VLSI Synthesis – Introduction & Complete Synthesis Flow	1
2	Deep Dive into Synthesis Flow – Part 1	2
3	Deep Dive into Synthesis Flow – Part 2	3
4	Boolean Optimization Techniques – Part 1	4
5	Boolean Optimization Techniques – Part 2	5
6	Synthesis Optimization Techniques – Part 1	6
7	Synthesis Optimization Techniques – Part 2	7
8	Additional Synthesis Optimization Techniques	8
9	RTL Design to Netlist Flow	9
10	Step-by-Step Synthesis Process in Cadence	10
11	Demonstration of Synthesis in Cadence – RTL to Gate-Level Netlist	11



Standard Cells
Macros
Power Structures
IO / Pads



PD SYNTHESIS

1. VLSI SYNTHESIS – INTRODUCTION :

VLSI Synthesis is the process of converting RTL (Verilog / VHDL) description into an optimized gate level netlist using a target standard cell library while meeting timing, area and power constraints.

2. EXAMPLE (COMPARE WITH PROGRAMMING) :

PROGRAMMING		VLSI SYNTHESIS
Write Code (C / Python)	→	Write RTL (Verilog / VHDL)
Compiler	→	Synthesis Tool (Genus / NC)
Object Code	→	Gate Level Netlist
Standard Library (CPU)	→	Standard Cell Library (ASIC)
Run Program	→	Implement Design in Silicon
Optimization (speed / size)	→	Optimization (timing / area / power)

3. COMPLETE SYNTHESIS FLOW :

1. RTL + INPUT LIBRARIES (RTL, .lib, .lef, .db, UPF, etc.)
2. ELABORATION (Hierarchy check, unconn. check)
3. READ SDC & SANITY CHECKS (Timing constraints, clocks, I/O)
4. GENERIC MAPPING (Map to generic gates)
5. TECHNOLOGY MAPPING (Map to target std. cells)
6. OPTIMIZATION (Timing, area, power opt.)
7. DFT INSERTION (Scan insertion, test logic)
8. POST DFT OPTIMIZATION (Re-optimize after DFT)
9. GATE LEVEL NETLIST (Synthesized netlist .v)

4. DESIGN INPUTS TO SYNTHESIS :

- RTL Source Files : Verilog / VHDL
- Standard Cell Library : .lib / .db
- Technology LEF : .lef
- Constraints (SDC) : clocks, I/O delays, timing, false paths, etc.
- UPF (Power Intent) : power domains, modes
- Other Files : scripts, config, etc.

5. KEY POINT :

* Synthesis creates a gate level netlist optimized for timing, area and power using the given libraries and constraints. This netlist is the starting point for Physical Design implementation.

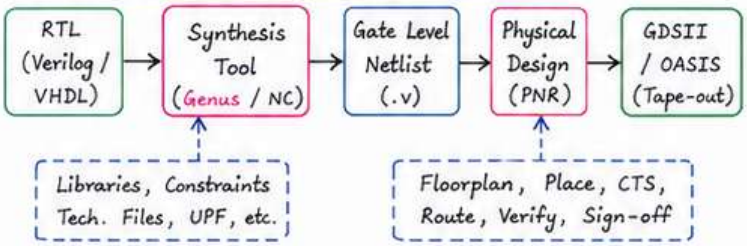
6. ADVANTAGES OF SYNTHESIS :

- ✓ Converts RTL to implementable hardware.
- ✓ Optimizes for timing, area and power.
- ✓ Reduces manual design effort.
- ✓ Ensures technology specific mapping.
- ✓ Supports DFT and low power requirements.

7. CHALLENGES :

- × Incomplete / wrong constraints → timing failures later.
- × High area or power if optimization is not done properly.
- × Library / tech mismatch → inaccurate results.
- × DFT insertion may impact area, timing, power.

8. SYNTHESIS FLOW DIAGRAM (OVERVIEW) :



9. GATE LEVEL NETLIST (OUTPUT) :

- Consists of standard cells and their interconnections.
- Contains no hierarchy (flattened).
- Used as input to Physical Design flow.
- File format : .v (Verilog)

10. SYNTHESIS VS PHYSICAL DESIGN :

Parameter	Synthesis	Physical Design
Input	RTL + Libraries + Constraints	Gate netlist + Tech files
Output	Gate level netlist (.v)	GDSII / OASIS
Main Goal	Optimize logic (timing, area, power)	Optimize physical implementation
Key Concern	Logic optimization	Timing closure, congestion, power, DRC, etc.
Tools	Genus / NC	ICC2 / Innovus
Sign-off	Reports (area, power, timing)	Timing, DRC, LVS, IR, EM

11. PRACTICAL OBSERVATIONS :

- Good constraints → good synthesis → better Physical Design.
- Early consideration of DFT & power simplifies implementation.
- Synthesis optimization directly impacts PPA and timing closure.

12. INTERVIEW / VIVA POINTS :

Q1. What is the purpose of synthesis ?
→ To convert RTL into an optimized gate level netlist using target technology while meeting timing, area and power constraints.

Q2. Why are constraints important in synthesis ?
→ Constraints guide the tool to optimize the design correctly. Wrong constraints lead to timing failures during Physical Design.

Q3. What is generated after synthesis ?
→ Gate level netlist (.v) along with reports (area, power, timing, cells, gates).

13. SUMMARY / REVISION POINTS :

* VLSI Synthesis converts RTL to gate level netlist.
* Flow : RTL → Elaboration → Constraints → Mapping → Optimization → DFT → Post-DFT Opt → Netlist.
* Inputs : RTL, Libraries, SDC, UPF, Tech files.
* Outputs : Gate netlist (.v) + reports (area, power, timing).
* Good synthesis ensures better results in Physical Design.
* Constraints are crucial for successful synthesis.
* Synthesis is the bridge between RTL design and PNR flow.

14. SIGN-OFF CHECKLIST (Post Synthesis Reports) :

- ✓ Area
- ✓ Power
- ✓ Timing (Setup / Hold)
- ✓ Cell Count
- ✓ Gate Count
- ✓ Unconstrained Paths
- ✓ Logic Levels
- ✓ DRC Pre-check (Logical)

NOTE : All reports must be reviewed before PNR.

PD SYNTHESIS

2. DEEP DIVE OF SYNTHESIS FLOW (STEP-BY-STEP)

RTL + INPUT LIBRARIES → ELABORATION → READ SDC & SANITY CHECKS → GENERIC MAPPING → TECHNOLOGY MAPPING

2.1 RTL + INPUT LIBRARIES :

- Synthesis starts with RTL description and required libraries / tech files.

Inputs	Description
RTL Files	Verilog / VHDL source code describing the design.
Standard Libraries (.lib)	Timing, power, area, pin & cell information of standard cells (target tech).
Technology Libraries (.lef / .tf / .db)	Technology parameters, routing layers, DRC info, via rules, etc.
Other Files	UPF (power intent), Constraints (SDC), Scripts, Configurations.

2.2 ELABORATION :

Definition :

- Elaboration is the process of reading RTL and resolving all hierarchy, parameters, generates to create a complete in-memory design database.

What internally happens ?

- Reads RTL files.
- Resolves module instances & hierarchy.
- Evaluates parameters and constants.
- Unrolls generate loops.
- Builds connectivity graph (nets, ports, instances).
- Checks basic syntax & semantic correctness.

Why it is important ?

- Creates a design database for further synthesis steps.
- Detects syntax / hierarchy issues early.
- Ensures design is complete and consistent.

✱ **KEY POINT :** Without correct elaboration, later steps (mapping, optimization) may produce wrong or incomplete results.

2.3 READ SDC & SANITY CHECKS :

What is SDC (Synopsys Design Constraints) ?

- SDC is a standard format file that defines timing, clocking, I/O, false paths, multicycle paths, uncertainties, driving cells, etc.

Example SDC Commands :

```
create_clock -name clk -period 2.5 [get_ports clk]
set_input_delay -max 1.0 -clock clk [get_ports in*]
set_output_delay -max 1.2 -clock clk [get_ports out*]
set_false_path -from [get_ports scan_in] -to [get_ports scan_out]
```

What is Sanity Check ?

- It is a quick verification to ensure constraints and design are reasonable before doing mapping.

Why we need this step ?

- Wrong / missing constraints → wrong optimization.
- Catches unconstrained paths, missing clocks, bad I/O definitions, etc.
- Ensures synthesis is guided correctly for timing, area & power.

Sanity Checks Include :

- ✓ All clocks defined ?
- ✓ All inputs / outputs have delays ?
- ✓ No unconstrained paths ?
- ✓ No floating ports ?
- ✓ False / multi-cycle paths verified ?

2.4 GENERIC MAPPING :

Definition :

Converts RTL constructs (always, if, case, operators, etc.) into technology independent generic gates.

Optimization in this step :

- Constant propagation & folding.
- Boolean simplification.
- Common sub-expression elimination.
- Logic restructuring.
- Resource sharing.
- Redundant logic removal.
- Input / output optimization.

Output :

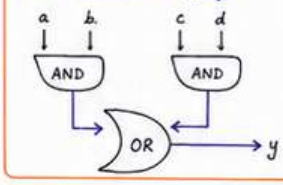
Generic gate level netlist (AND, OR, XOR, MUX, DFF, etc.)

Example :

RTL :

```
assign y = (a & b) | (c & d);
```

After Generic Mapping :



2.5 TECHNOLOGY MAPPING :

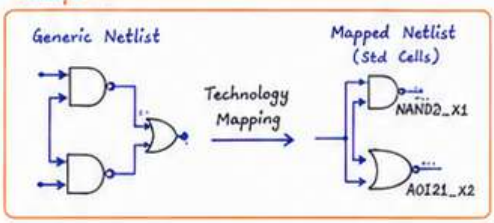
Definition :

Maps generic gates to actual standard cells from the target technology library.

What happens internally ?

- Each generic gate is replaced with an equivalent cell from the .lib.
- Considers timing, area, power, drive strength, pin capacities.
- Cell selection based on optimization cost function.
- Balancing of logic levels and fanout.

Example :



Applications / Uses :

- Converts abstract design to implementable form.
- Prepares for timing, power & area estimation.
- Required before Physical Design implementation.

COMPARISON : GENERIC MAPPING vs TECHNOLOGY MAPPING

Parameter	Generic Mapping	Technology Mapping
Input	Elaborated RTL + Constraints	Generic Gate Netlist + Libraries
Output	Generic Gate Netlist	Mapped Gate Level Netlist
Gate Types	Technology Independent (AND, OR, XOR, MUX, DFF...)	Technology Dependent (Std Cells from .lib)
Optimization	Logic optimization, Boolean simplification, resource sharing	Cell selection, drive strength, Timing / Power / Area optimization
Library Used	Not required	Standard Cell Library (.lib)
Awareness	Timing, area (approx)	Timing, area, power (accurate)
Purpose	Create optimized abstract logic	Create implementable netlist

PRACTICAL OBSERVATIONS :

- Good constraints → better mapping and optimization.
- Earlier constraint checks save lot of re-work later.
- Technology mapping cost function balances timing, area, power.
- Bad constraints cause pessimistic or optimistic results.

INTERVIEW / VIVA POINTS :

Q1. Why is elaboration important in synthesis ?
→ It builds the complete design database and resolves hierarchy for further processing.

Q2. Difference between generic and technology mapping ?
→ Generic mapping creates technology independent gates, while technology mapping maps them to actual standard cells.

PD SYNTHESIS

1. OPTIMIZATION :

Definition :

Optimization improves the quality of the synthesized design to meet timing, area and power goals while satisfying all constraints.
It is an iterative process throughout synthesis.

Types of Optimization :

TIMING OPTIMIZATION	AREA OPTIMIZATION	POWER OPTIMIZATION
• Reduce path delay • Improve slack • Fix timing violations • Balance path delays	• Reduce cell count • Share logic • Resource sharing • Remove redundant logic	• Reduce switching activity • Lower leakage • Clock gating • Resource control

Optimization Steps (Typical) :

1. Check timing and constraint violations.
2. Identify critical paths / high switching nets.
3. Apply transformations (logic rewrite, restructuring).
4. Use cell upsizing / downsizing / buffering.
5. Retiming and path balancing (useful for timing).
6. Remove redundant logic & constant propagation.
7. Re-run timing, area and power analysis.
8. Iterate until goals are met.

KEY POINT : Optimization is not a single step, it is applied multiple times across the flow (pre DFT and post DFT).

2. DFT INSERTION :

Definition :

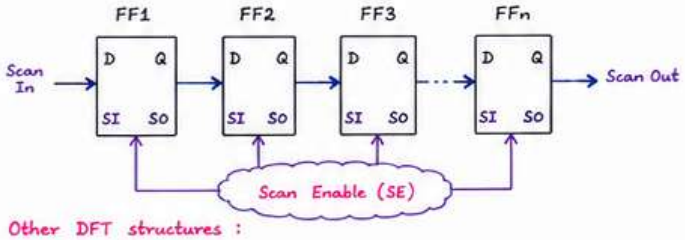
DFT (Design For Test) adds test structures so that the design can be tested effectively after fabrication.

Goal :

Increase controllability and observability of internal nodes.

2.1 Scan Insertion :

- Flip-flops are replaced with scan flip-flops.
- Scan chains are created to shift in test vectors and shift out responses.



Other DFT structures :

- Test points
- Clock gating cells
- BIST (Built-In Self Test) logic (optional)
- MBIST / LBIST
- Boundary Scan (for chip level test)

How to improve controllability ?

- Increase number of scan cells
- Add test points on hard to control nodes
- Break long combinational paths
- Use constant / partial scan

How to improve observability ?

- Add test points near primary outputs
- Add scan on outputs of internal logic
- Use compression / decompression logic
- Add observation points for critical buses

Better C & O → Better Testability → Higher Fault Coverage

3. POST DFT OPTIMIZATION :

Definition :

After inserting DFT structures, the design area, timing and power may degrade. Post DFT optimization restores the quality while keeping testability intact.

Usages / Why it is needed ?

- Scan insertion increases area, delay and power.
- Post DFT opt. removes redundant logic introduced.
- Fix new timing violations caused by DFT logic.
- Reduce switching activity from scan logic.
- Achieve timing, area and power closure with test.

Typical Steps :

1. Re-analyze timing after DFT insertion.
2. Fix timing violations (re-buffer, re-size, re-time).
3. Remove redundant / equivalent logic.
4. Clock gating / scan gating for power reduction.
5. Re-optimize area and power.
6. Validate with constraints and DRC.

FLOW AROUND DFT :

Optimization → DFT Insertion → Post DFT Optimization → Gate Level Netlist

KEY POINT : DFT insertion improves testability, while Post DFT Optimization recovers the design quality.

4. GATE LEVEL NETLIST :

Definition :

Gate level netlist is the final output of synthesis. It describes the design using technology specific standard cells.

Contents :

- Cell Instances → AND, OR, NAND, flops, mux, etc.
- Connections (Nets) → Between pins and cells
- Cell Types → From target technology library
- Hierarchy → Modules / sub-modules
- Timing Information → Delays, arc info (implicit)
- Constraints Applied → Reflected in the structure
- Scan Chains / DFT cells → If DFT inserted
- Ports → Primary inputs & outputs
- Power / Ground nets → VDD, VSS connections
- Attributes → Drive strength, area, etc.

File Format : .v (Verilog) or .vg / .vhdl

Usage in Physical Design :

- Input to Physical Design flow (P&R).
- Used for floorplanning, placement, CTS, routing.
- Basis for timing analysis with parasitics.
- Used for equivalence checking and sign-off.

SUMMARY COMPARISON

Stage	Main Purpose	Effect	Key Output
Optimization	Improve timing, area, power	Better PPA	Optimized netlist
DFT Insertion	Improve Testability	Adds scan, test pts	DFT inserted netlist
Post DFT Opt.	Recover PPA with test	Fix timing, reduce area/pwr	Optimized DFT netlist
Gate Level Netlist	Final logic netlist	Ready for P&R	.v netlist (gate level)

INTERVIEW / VIVA POINTS :

Q1. Why is Post DFT Optimization required ?
→ DFT logic increases area and delays, post DFT opt. restores timing, area and power while keeping testability.

Q2. What is the main goal of DFT insertion ?
→ To improve controllability and observability for high fault coverage.

Q3. What is the usage of Gate Level Netlist in PD ?
→ It is the primary input for P&R, STA and sign-off.

PD SYNTHESIS

1. BOOLEAN OPTIMIZATION TECHNIQUES :

Definition :

Boolean optimization reduces the complexity of a logic network by using mathematical identities and structural analysis.

Objectives :

- ✓ Reduce gate count (area)
- ✓ Improve timing (reduce critical path delay)
- ✓ Reduce switching activity (power)
- ✓ Simplify logic for easier mapping & better testability

1.1. BOOLEAN ALGEBRIC SIMPLIFICATION :

Definition :

Uses Boolean algebra laws and identities to simplify expressions and reduce number of literals and terms.

Steps :

1. Write the Boolean expression.
2. Apply Boolean algebra laws (Identity, Null, Idempotent, Absorption, Distributive, DeMorgan's, Consensus, etc.)
3. Remove redundant literals / terms.
4. Factor / rewrite for minimum form.
5. Implement the simplified expression.

Advantages :

- Simple and fast.
- No extra hardware.
- Works at early stage.
- Reduces area and delay.

Disadvantages :

- Manual simplification may miss optimal solution.
- Becomes difficult for large expressions.
- Doesn't consider physical effects.

Example :

Simplify : $F = AB + AC + BC$

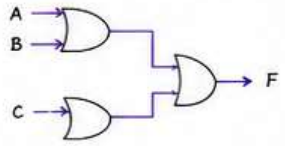
Solution :

$F = AB + AC + BC$

$= A(B + C) + BC$ — factor A

$= A(B + C) + BC$

$= (A + B)(A + C)$ — using $(X + Y)(X + Z) = X + YZ$



1.2. COMMON SUB EXPRESSION ELIMINATION (CSE) :

Definition :

Detects and shares logic that is used multiple times in the expression.

Steps :

1. Identify repeated sub-expressions.
2. Create a temporary node for the sub-expression.
3. Replace all occurrences with the shared node.
4. Re-synthesize the logic.
5. Optimize the remaining logic.

Advantages :

- Reduces gate count.
- Reduces power (less switching).
- Improves timing.
- Improves area efficiency.

Disadvantages :

- Extra fanout on shared logic.
- May increase load and delay.
- Not always beneficial if sharing causes congestion.

Example :

$F = AB + AC + AD + ABC$

Step 1: Common sub-expression = A

Step 2: Let $X = A$

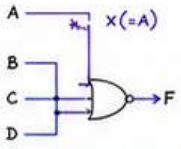
$F = AX + B\bar{X}C + DX + AX\bar{B}C$

$= X(B + C + D) + XBC$ where $X = A$

$= X[(B + C + D) + BC]$

$= X(B + C + D)$ (since $(B+C+D) + BC = B+C+D$)

$F = A(B + C + D)$



SUMMARY POINTS :

1. Optimization improves area, timing and power before DFT.
2. DFT insertion adds test structures to improve testability.
3. Post DFT optimization restores design quality.
4. Gate level netlist is the final output for Physical Design flow.

Sharing saves

- 2 AND gates
- 1 input of AND
- Power & delay

1.3. REDUNDANT LOGIC REMOVAL :

Definition :

Removes logic that does not affect the output under any input condition.

Steps :

1. Analyze logic for redundancy.
2. Remove redundant gates / branches / terms.
3. Use don't care conditions (if available).
4. Re-synthesize and optimize.

Advantages :

- Reduces area and delay.
- Reduces power.
- Simplifies design.
- Improves reliability.

Disadvantages :

- Incorrect removal may change functionality.
- Requires careful verification.
- Tool dependent accuracy.

Example :

$F = AB + A\bar{B}C + A\bar{B}C$

Solution :

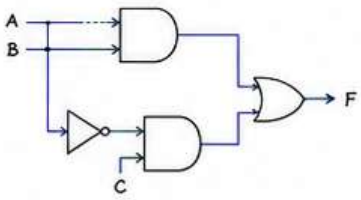
AB is common in all terms.

$F = AB(1 + \bar{C} + C)$

$= AB(1)$ — since $(X + \bar{X}) = 1$

$= AB$

$F = AB$



KEY POINT :

Combination of algebraic simplification, CSE and redundant logic removal leads to minimum gate count, better timing and lower power before mapping.

2. DFT INSERTION :

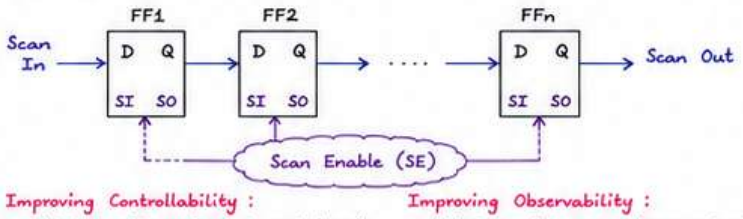
Definition :

DFT (Design For Test) insertion adds test structures (scan chains, test points, BIST, etc.) to improve testability.

Goal : Improve controllability and observability of internal nodes.

Scan Insertion :

- Replace all or selected flip-flops with scan flip-flops.
- Connect scan FFs in series to form scan chains.



Improving Controllability :

- Add test points on hard to control nodes.
- Use scan enable to force states.
- Use constant inputs where possible.
- Break long combinational paths.

Improving Observability :

- Add test points near primary outputs.
- Use scan chains to capture states.
- Add observation points on bus / nodes.
- Use compression / decompression logic.

Other DFT Structures :

- Test Points (TP)
- Clock Gating Cells
- BIST / LBIST
- Boundary Scan (for chip level test)

Benefits of DFT :

- ✓ High fault coverage
- ✓ Easier testing
- ✓ Reduced test time
- ✓ Early defect detection.

3. POST DFT OPTIMIZATION :

Definition :

Optimizatiog after inserting DFT to recover area, timing and power degradation caused by test logic.

Usages :

- Remove redundant logic introduced by DFT.
- Buffer insertion / resizing for timing.
- Re-balance logic for area and power.
- Fix new timing violations.

4. GATE LEVEL NETLIST :

Definition :

A technology mapped netlist consisting of standard cells and their interconnections.

Contents :

- Standard Cell Instances
- Nets / Connections
- Cell Types
- Timing info (if annotated)
- Constraints Applied

Usage in Physical Design :

- Input to Floorplanning
- Used for Placement, CTS, Routing
- Basis for STA & Sign-off
- Used for Parasitic Extraction
- Constraints Applied

PD SYNTHESIS

4. CONSTANT PROPAGATION :

Definition :

Constant propagation replaces variables with known constant values (0 or 1) and simplifies the logic network.

Steps :

1. Identify signals / inputs with constant values (0 or 1).
2. Substitute the constant in the logic expression or circuit.
3. Simplify the expression / remove redundant gates.
4. Iterate until no more simplification is possible.

Advantages :

- Reduces gate count and area.
- Improves timing.
- Simplifies logic for mapping.
- Lowers switching activity.

Disadvantages :

- Depends on availability of constant values.
- May not always be possible at early stages.

Example :

Given : $F = A\bar{B} + C\ (A + 0) + D\ (1)$

Step 1 : Substitute constants

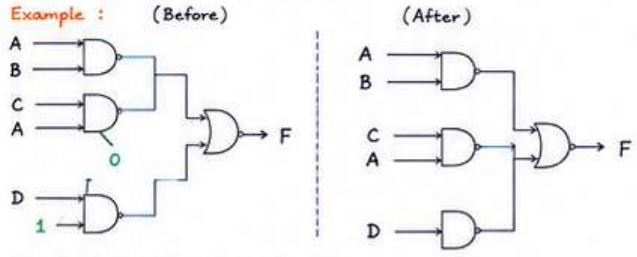
$F = A\,B + C\ (A) + D$

$= A\,B + A\,C + D$

Step 2 : Factorize

$F = A\ (B + C) + D$

Circuit Example :



5. DEMORGAN'S TRANSFORMATION :

Definition :

DeMorgan's theorems are used to transform AND ↔ OR and complement the literals to simplify logic or to meet implementation requirements (e.g., NAND / NOR).

Theorems :

1. $(A + \bar{B})' = A'\,B'$ (Complement of OR = AND of complements)
2. $(A\,\bar{B})' = A' + B'$ (Complement of AND = OR of complements)

Steps :

1. Identify the expression / sub-expression.
2. Apply DeMorgan's theorem.
3. Simplify if possible.
4. Repeatedly apply to reach desired form.

Advantages :

- Enables implementation using only NAND or NOR gates.
- Helps in logic minimization.
- Useful in multi-level optimization.

Disadvantages :

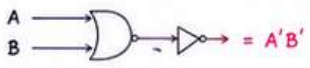
- May increase number of literals.
- Can introduce extra inversions.
- May not always reduce area.

Example :

1. $F = (A + B)'$

Using DeMorgan :

$F = A'\,B'$

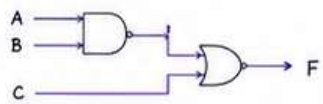


2. $F = (A\,B)' + C$

Using DeMorgan :

$F = (A' + B') + C$

$= A' + B' + C$



KEY POINT : DeMorgan's transformation is frequently used in synthesis to match logic with gate libraries (NAND/NOR based) and to push inversions for better optimization.

6. KMAPS (KARNAUGH MAPS) :

Definition :

K-Map is a graphical method used to minimize Boolean expressions by grouping adjacent 1's (or 0's) considering the don't care conditions.

Steps :

1. Write the Boolean expression in minterm / maxterm form.
2. Draw the K-Map (2, 3, 4, 5, ,... variables).
3. Fill the cells with 1's (or 0's) and X's (don't cares).
4. Group adjacent 1's in powers of 2 (1, 2, 4, 8, ...) using horizontal, vertical and wrap-around adjacency.
5. Write simplified expression for each group.
6. Combine all groups to get minimal expression.

Advantages :

- Simple and visual.
- Gives minimal expression.
- Handles don't care conditions.
- Better for small number of variables (≤ 6).

Disadvantages :

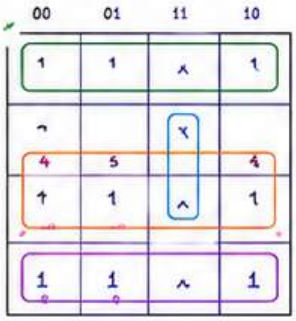
- Not practical for > 6 variables.
- Manual method is time consuming for large functions.
- Error prone for many variables.

Example : Minimize $F(A,B,C,D) = \Sigma m(0,1,2,5,6,7,8,9,10,14) + d(11,15)$

Step 1 : 4-variable K-Map

AB \ CD	00	01	11	10
00	1 (0)	1 (1)	X (3)	1 (2)
01	0 (4)	1 (5)	X (7)	1 (6)
11	1 (12)	1 (13)	X (15)	1 (14)
10	1 (8)	1 (9)	X (11)	1 (10)

Step 2 : Grouping



Step 3 : Expression for each group

→ Group 1 (top row) → $A'B'$ (A=0, B=0)

→ Group 2 (bottom row) → AB (A=1, B=1)

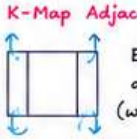
→ Group 3 (left column) → $B'D'$ (B=0, D=0)

→ Group 4 (column CD=10) → $B\,D'$ (B varies, D=0)

Final Simplified Expression :

$F = A'B' + AB + B'D' + B\,D'$

K-Map Adjacency :



Edges are adjacent (wrap-around)

KEY POINT :

- K-Map minimizes the number of literals and hence gates, which leads to better area, timing and power after synthesis.

BOOLEAN OPTIMIZATION TECHNIQUES – SUMMARY

Technique	Purpose	Best Used When	Impact in PD Synthesis
Const. Propagation	Replace vars with constants	Some inputs / outputs are constant	Reduces gates, improves timing & power
DeMorgan's Transformation	Complement change & OR ↔ AND conversion	Need NAND/NOR implementation	Matches library, better mapping
K-Maps	Minimize Boolean expressions	≤ 6 variables (small functions)	Reduces area, improves performance

INTERVIEW / VIVA QUESTION :

Q. Why Boolean optimization is important in PD synthesis ?

→ It reduces gate count, area, power and critical path delay, leading to better PPA and easier physical implementation.

PD SYNTHESIS

SYNTHESIS OPTIMIZATION TECHNIQUES :

Definition :

Optimization in synthesis is the process of improving a design to achieve better area, timing and power without changing its functionality.

Objectives :

- ✓ Reduce area (fewer gates / cells).
- ✓ Improve timing (reduce delay, meet constraints).
- ✓ Reduce power (switching activity, leakage).
- ✓ Remove redundant logic & logic that does not affect output.
- ✓ Improve design quality and reliability.

1. TECHNOLOGY INDEPENDENT OPTIMIZATION :

Definition :

Optimization performed before mapping to a specific technology. Works on Boolean level / graph level to simplify the design.

Characteristics :

- Uses Boolean algebra & logic restructuring.
- Not dependent on target standard cell library.
- Reduces complexity before technology mapping.
- Helps in better area, timing and power later.

Common Techniques :

- Boolean algebraic simplification
- Common sub-expression elimination
- Redundant logic removal
- Constant propagation
- DeMorgan's transformations
- K-Map based simplification

Steps :

1. Read RTL & constraints.
2. Elaborate and build logic network.
3. Apply TI optimizations on the network.
4. Recompute and propagate changes.
5. Verify equivalence (no functional change).
6. Pass optimized network to technology mapping.

Advantages :

- Reduces gate count & logic depth.
- Improves timing and power.
- Produces better results for technology mapping.
- Faster synthesis runtime in later stages.

Disadvantages :

- May not consider real cell delays or drive strengths.
- Over-simplification may increase fanout.
- Doesn't consider physical aspects (load, wiring).

Example :

Boolean simplification : $F = AB + \bar{A}C + AC$

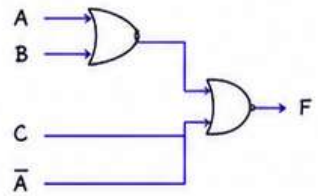
Step 1: Factor A from first and third term.

$$F = A(B + C) + \bar{A}C$$

Step 2: No further common term.

Final optimized expression :

$$F = A(B + C) + \bar{A}C$$



2. DEAD LOGIC ELIMINATION :

Definition :

Removal of logic or circuitry that does not affect the primary outputs for any input combination.

Why Dead Logic Occurs ?

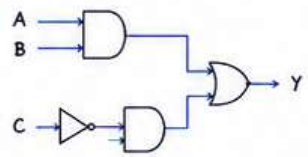
- Unconnected outputs.
- Logic driving constants (0/1) that do not influence output.
- Unused inputs or logic paths.
- Redundant or unreachable logic.

Steps :

1. Perform logic traversal from primary outputs.
2. Mark all reachable nodes.
3. Remove all nodes / gates not marked (unreachable).
4. Reconnect and simplify the remaining network.
5. Verify equivalence.

Example :

Circuit :



Observation :
The lower path (C → inverter → AND gate) output is not used anywhere → dead logic.

After Dead Logic Elimination :



Advantages :

- Reduces area.
- Reduces power.
- Improves timing.
- Simplifies verification.

Disadvantages :

- Need full logic traversal.
- May remove logic if constraints are incomplete.
- Care needed in sequential logic (sequential don't cares).

COMPARISON TABLE

Aspect	Technology Independent Optimization	Dead Logic Elimination
When	Early in synthesis flow	After elaboration / TI opt.
Dependence	Independent of target library	Independent of target library
Purpose	Simplify logic network	Remove unreachable logic
Techniques	Boolean simplification, CSE, Redundant logic, Constant prop., DeMorgan's, K-Map	Reachability analysis, Logic pruning
Impact	Improves area, timing, power before mapping	Removes useless gates, reduces area & power
Risk	Over-simplification may increase fanout	May remove logic if constraints are incorrect
Output	Optimized logic network	Pruned logic network
Verification	Equivalence check required	Equivalence check required

SUMMARY POINTS :

1. Optimization is essential to achieve better PPA and QoR.
2. Technology Independent Optimization works on Boolean level.
3. Dead Logic Elimination removes unreachable or unused logic.
4. Both techniques reduce gate count, improve timing and power.
5. Equivalence checking is mandatory after every optimization.

INTERVIEW / VIVA QUESTIONS :

Q1. What is Technology Independent Optimization in synthesis ?
→ Optimization done before mapping to a specific library using Boolean level techniques.

Q2. What is Dead Logic Elimination ?
→ Removal of logic that does not affect outputs for any input.

Q3. Why is dead logic harmful if not removed ?
→ It increases area, power and may affect timing.

... PD SYNTHESIS

SYNTHESIS OPTIMIZATION TECHNIQUES

3. LOGIC RESTRUCTURING

- Logic restructuring modifies the logic implementation without changing functionality to improve timing, area, power and routability.
- Achieved by Boolean transformations, gate reordering, factoring, decomposition and equivalent restructuring.
- Helps to reduce logic depth, critical path delay and switching activity.

Techniques :

1. Boolean factoring (common sub-expression sharing)
2. Algebraic transformations (DeMorgan, associativity)
3. Reordering of operations for lower delay
4. Gate decomposition (complex → simple gates)
5. Eliminate redundant logic and duplicate gates
6. Balance logic paths to reduce worst-case delay

Benefit :

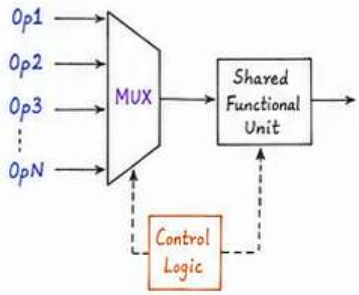
- Reduced critical path delay
- Lower power due to reduced switching
- Better timing closure
- Improved area and routability
- Reduced logic depth and fanout

4. RESOURCE SHARING

- Resource sharing allows multiple operations to use the same hardware resource over time.
- Especially useful in area/power constrained designs.
- Achieved using multiplexers, control logic and scheduled operations.

Techniques :

1. Functional unit sharing (ALU, multiplier, etc.)
2. Time multiplexing of resources
3. Operator sharing in datapaths
4. Memory & bus sharing



Benefit :

- Reduces area and power
- Lower silicon cost
- Efficient utilization of hardware
- Controlled trade-off between area and performance

5. DON'T CARE OPTIMIZATION

- Don't care conditions allow flexibility in logic simplification and optimization.
- Signals or states that never occur or are irrelevant for correct operation.
- Used to reduce logic gates and improve implementation.

Sources of Don't Cares :

1. Unreachable states
2. Invalid input combinations
3. Output don't cares
4. Redundant conditions

Example :

AB \ C	00	01	11	10
0	1	0	X	1
1	X	1	0	X

X → Don't care

Can be used to simplify logic further.

Benefit :

- Fewer gates → lower area
- Reduced switching → lower power
- Shorter critical paths → better timing
- Simpler logic → better routability

HOW THEY IMPROVE PD SYNTHESIS ?

- Improve logic quality before placement & routing.
- Reduce critical path and logic depth.
- Lower cell count and switching activity.
- Reduce congestion by simplifying logic.
- Help meet timing, power and area targets.

Technique	Focus	Main Impact	Benefit in PD
Logic Restructuring	Modify logic structure	Reduce delay & logic depth	Better timing & routability
Resource Sharing	Share hardware resources	Reduce area & power	Low area, low power designs
Don't Care Optimization	Use don't cares for simplification	Fewer gates & nets	Better area, power & timing

CONCEPTUAL COMPARISON

Parameter	Logic Restructuring	Resource Sharing	Don't Care Optimization
Goal	Improve logic structure	Share hardware resources	Use don't cares for simplification
Method	Boolean transformations	Time multiplexing & control	Use unreachable / invalid conditions
Impact	Delay, depth, routability	Area, power reduction	Area, power, timing
Area	Moderate Improvement	High Improvement	Moderate to High
Power	Reduced switching	Reduced activity	Reduced switching
Use Case	Performance critical paths	Area / power constrained	Logic simplification

PRACTICAL OBSERVATIONS

1. Logic restructuring is applied early in synthesis for better downstream P&R results.
2. Resource sharing is essential in low-power and area constrained designs.
3. Don't care optimization heavily depends on correct identification of unreachable states.

> ☆ KEY POINT ☆
>
> These optimization techniques enhance the synthesized netlist quality before physical implementation, leading to better timing closure, lower power, reduced area and improved routability.

INTERVIEW / VIVA POINTS

Q① Why is logic restructuring important in PD synthesis ?

→ It reduces logic depth and critical path delay, improving timing closure and routability.

Q② How does resource sharing help in low power designs ?

→ By sharing hardware resources, switching activity and area are reduced, leading to lower power.

SUMMARY / REVISION POINTS

- Logic restructuring modifies logic to reduce delay and depth.
- Resource sharing uses same hardware for multiple operations over time.
- Don't care optimization uses irrelevant conditions to simplify logic.
- These techniques improve area, power, timing and routability.
- Early optimization leads to better physical implementation and timing closure.

PD IMPACT METRICS

- Area ↓
- Power ↓
- Delay ↓
- Logic Depth ↓
- Switching Activity ↓
- Congestion ↓
- Timing Slack ↑
- Routability ↑

PD SYNTHESIS

8. ADDITIONAL SYNTHESIS OPTIMIZATION TECHNIQUES :

These techniques are used by the synthesizer to improve area, timing, power and design quality beyond basic optimizations.

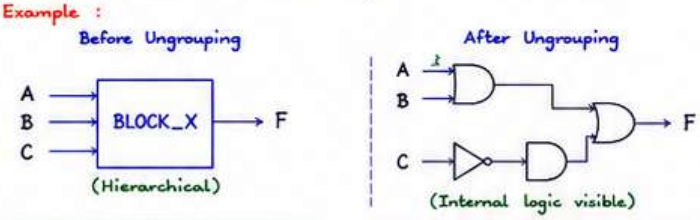
KEY OBJECTIVES :
- ✓ Reduce area and power.
- ✓ Improve timing (setup / hold).
- ✓ Increase design flexibility and QoR.
- ✓ Enable better technology mapping and placement.

1. UNGROUPING :

Definition :
Breaking a hierarchical block (module) and bringing its internal logic to the top level to expose more optimization opportunities.

Benefits :
- Enables cross-boundary optimizations.
- Removes unnecessary logic.
- Improves area and timing.
- Helps in better resource sharing.

Example :



Advantages :
- Better optimization.
- Removes redundant logic.
- Improves timing.

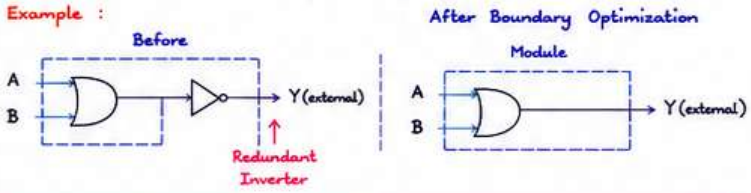
Disadvantages :
- Increases design size.
- May increase compile time.
- May break reusability.

2. BOUNDARY OPTIMIZATION :

Definition :
Optimizing logic at the interface (boundary) between modules or between the design and its environment.

Benefits :
- Eliminates redundant logic at boundaries.
- Improves timing and reduces area.
- Reduces loading / fanout at interface.
- Better handshake and signal alignment.

Example :



Advantages :
- Removes redundant interface logic.
- Improves timing margins.
- Reduces power and area.

Disadvantages :
- Requires visibility of boundary.
- May need constraints area.

3. PATH GROUPING :

Definition :
Grouping related paths (critical / non-critical) and optimizing them together based on timing goals and constraints.

Benefits :
- Helps in balancing multiple paths.
- Improves overall timing QoR.
- Ensures timing closure with less area.
- Better control over critical vs non-critical paths.

Path Group
- Critical Path Group
- Non-critical Path Group
- Asynchronous Group
- False Path Group

Advantages :
- Better timing closure.
- Balanced resource usage.
- Avoids over-optimization.

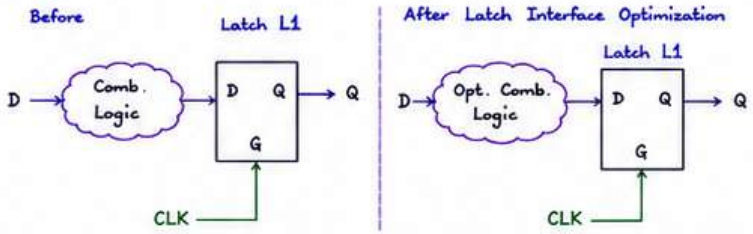
Disadvantages :
- Incorrect grouping may degrade timing.
- Needs accurate constraints.

4. LATCH INTERFACE OPTIMIZATION :

Definition :
Optimizing logic around latch boundaries to improve timing (setup/hold) and reduce glitches.

Benefits :
- Improves setup and hold timing.
- Reduces unnecessary toggle / glitches.
- Balances latch transparency windows.
- Reduces power.

Example :



Advantages :
- Better latch setup/hold.
- Fewer glitches.
- Lower power switching.

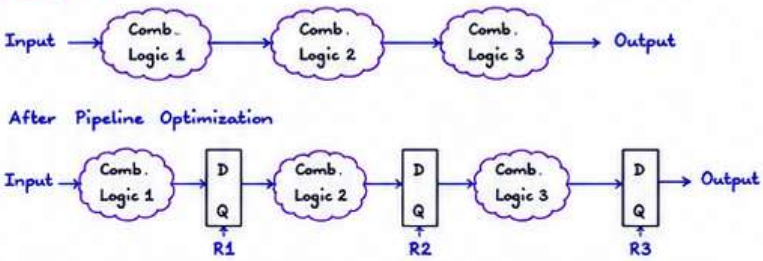
Disadvantages :
- May increase logic on critical path.
- More complex optimization.

5. PIPELINE OPTIMIZATION :

Definition :
Inserting, removing or relocating pipeline registers to improve performance, balance stages and reduce latency or power.

Benefits :
- Improves throughput.
- Reduces critical path delay.
- Balances pipeline stages.
- Better power-performance trade-off.

Example :



Advantages :
- Higher clock frequency.
- Better timing closure.
- Stage balancing.
- Reduced logic delay.

Disadvantages :
- Increases latency.
- More registers (area/power).
- Complex control.

SUMMARY POINTS :
1. Ungrouping exposes internal logic for better optimization.
2. Boundary optimization removes redundant interface logic.
3. Path grouping balances and optimizes multiple paths.
4. Latch interface optimization improves setup/hold and reduces glitches.
5. Pipeline optimization increases performance and balances stages.

INTERVIEW VIVA QUESTIONS :

Q1. Why is ungrouping used in synthesis ?
→ To expose internal logic for better cross-boundary optimization and to remove redundant logic.

Q2. What is boundary optimization ?
→ It optimizes logic at module or design interfaces to improve timing and reduce area.

Q3. What is the benefit of pipeline optimization ?
→ It improves performance by reducing critical path delay and balances the pipeline stages.

PD SYNTHESIS

1. SYNTHESIS IN PHYSICAL DESIGN

- PD Synthesis converts the verified RTL into a gate level netlist using a target technology library and constraints.
- The synthesized netlist is the starting point for Physical Implementation.
- Objective → meet Timing, Area, Power targets.

2. RTL TO NETLIST FLOW

System Specification
↓
Architecture Design
↓
RTL Design ← t_B, Formal Verification
↓
Verified RTL
↓
Synthesis Inputs
- RTL Files (.v / .sv)
- Liberty (.lib)
- Constraints (.sdc)
- Tech files (lef, tf)
- UPF (power intent)

↓
Write Synthesis TCL Scripts
↓
Launch Tool (e.g. Synopsys DC)
↓

OPERATIONS PERFORMED BY TOOL

- Read RTL
- Parsing & Elaboration
- Design Rule Check
- Link Libraries
- Read Constraints
- Logic Optimization
- Technology Mapping
- Synthesis Optimization
- Static Timing Analysis
- QOR Optimization
  - Area
  - Power
  - Timing
- DRC (logical)
- Generate Reports

↓
SYNTHESIS OUTPUTS

Gate Level Netlist (.v)	Updated Constraints (.sdc)	Timing Report (.rpt)	Area Report (.rpt)	Power Report (.rpt)

3. KEY INPUTS EXPLAINED

- Liberty (.lib) → cell timing, power, area information.
- Technology Files → tech LEF / tf for layers, vias, rules.
- Constraints (.sdc) → clocks, I/O, timing exceptions.
- UPF → power domains, supply sets.
- RTL Files → synthesizable Verilog / SystemVerilog.

KEY POINT

Good quality synthesis depends on accurate constraints, complete libraries and clean RTL.
Synthesized netlist quality directly impacts downstream Physical Design performance.

4. SAMPLE SYNTHESIS SCRIPT STEPS (TCL)

1. read_verilog <rtl_files>
2. elaborate
3. link
4. read_liberty <lib_files>
5. read_sdc <constraints>
6. set_top <top_module>
7. compile_ultra (or compile)
8. write -format verilog -output <netlist.v>
9. report_timing, report_area, report_power

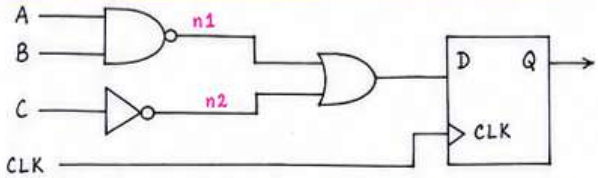
5. LOGICAL SYNTHESIS CONSIDERATIONS

- Timing Driven → optimize for setup & hold.
- Area Driven → minimize gate count.
- Power Driven → reduce switching & leakage.
- Multi-corner, Multi-mode (MCMM) analysis.
- Don't care conditions → better optimization.
- Retiming and resource sharing for performance.

6. SYNTHESIS QUALITY METRICS (QOR)

Metric	Description	Goal
WNS (Setup)	Worst Negative Slack	> 0
TNS (Setup)	Total Negative Slack	= 0
WHS (Hold)	Worst Hold Slack	> 0
Area	Total cell area	Minimize
Power	Total dynamic + leakage	Minimize

7. SYNTHESIZED NETLIST EXAMPLE (ABSTRACT)



(Actual synthesis optimizes and maps to library cells)

8. SYNTHESIS ISSUES

- Missing constraints → incorrect optimization.
- Over constrained design → infeasible timing.
- Incomplete libraries → sub-optimal mapping.
- Poor RTL coding → large area, poor timing.
- Ignoring hold → post-layout hold violations.

9. SYNTHESIS SIGN-OFF CHECKLIST

- ✓ All constraints applied
- ✓ No unconstrained paths
- ✓ Timing met (WNS > 0)
- ✓ Area within budget
- ✓ Power within budget
- ✓ No combinational loops
- ✓ No high fanout issues
- ✓ Reports reviewed
- ✓ Netlist lint clean
- ✓ Ready for PNR

10. VIVA / INTERVIEW POINTS

Q1. Why is synthesis important in PD?
→ It converts RTL to gate level netlist optimized for timing, area and power using target technology.

Q2. What affects synthesis quality?
→ Constraints, libraries, RTL coding style and optimization strategy.

11. SUMMARY / REVISION POINTS

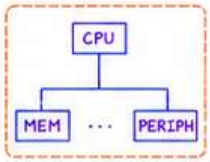
- RTL → verified → synthesis.
- Inputs → RTL, libs, constraints, tech files, UPF.
- Tool performs optimization, mapping and analysis.
- Outputs → netlist, updated constraints, timing, area, power reports.
- Good synthesis is foundation for successful Physical Design.

PD SYNTHESIS

10. STEP BY STEP PROCESS IN CADENCE TOOL :

① Architecture Design :

- Define system architecture, datapath, control.
- Partitioning, clocking strategy, power intent.
- Decide performance, area, power targets (PPA).
- High level block diagram & interface definition.



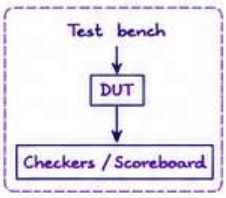
② RTL Design :

- Write RTL code in Verilog / SystemVerilog.
- Follow coding guidelines for synthesis.
- Modular, parameterized & reusable code.
- Add assertions for better verification.

```
module adder (a,b,sum);
 input [7:0] a,b;
 output [7:0] sum;
 assign sum = a + b;
endmodule
```

③ Test bench & Functional Verification :

- Develop test bench to verify functionality.
- Apply constrained random / directed tests.
- Check corner cases & error conditions.
- Achieve 100% functional coverage (goal).



④ Prepare Synthesis Inputs (All required files) :

i)	RTL Files	: .v / .sv	→ Source RTL
ii)	Liberty Files	: .lib	→ Standard cell libs
iii)	LEF Files	: .lef	→ Physical info (cells)
iv)	SDC File	: .sdc	→ Timing constraints
v)	UPF File	: .upf	→ Power intent
vi)	Tech Files	: Tech LEF, Milkyway libs etc.	

Purpose :
- Provide tool with logical, timing, physical & power information.
- Ensure correct mapping & optimization.

⑤ Automate the Process using TCL Script :

- Write TCL script to control synthesis flow.
- Set libraries, constraints, options, reports.
- Ensures repeatability, consistency & efficiency.
- Easier to debug and maintain large flows.

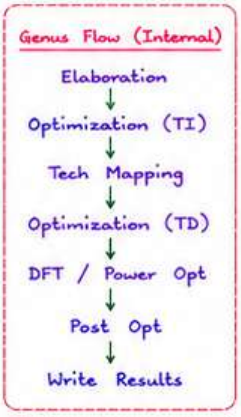
Sample TCL Commands :

```
read_verilog {rtl/*.v}                          # Read RTL
read_liberty {lib/typical/*.lib}                # Read Libraries
read_lef {lef/tech.lef lef/cells.lef}           # Read LEF files
read_sdc {constraints/top.sdc}                  # Read constraints
read_upf {power/top.upf}                        # Read Power intent
set_top top_module                              # Set top module
synth                                           # Run synthesis
write -format verilog -hierarchy -output netlist.v  # Write netlist
report_timing > reports/timing.rpt              # Timing report
report_area   > reports/area.rpt                # Area report
report_power  > reports/power.rpt               # Power report
exit                                            # Exit
```

⑥ Cadence Performs Internal Tasks using Genus :

- Genus is the synthesis engine in Cadence.
- Performs elaboration, optimization & mapping.
- Major internal tasks :

1. Read & elaborate design.
2. Technology independent optimization.
3. Technology mapping to target library.
4. Logic optimization (timing/area/power).
5. Clock gating, power optimization.
6. DFT (scan insertion) if enabled.
7. Post synthesis optimizations.



⑦ The Tool Generates Outputs :

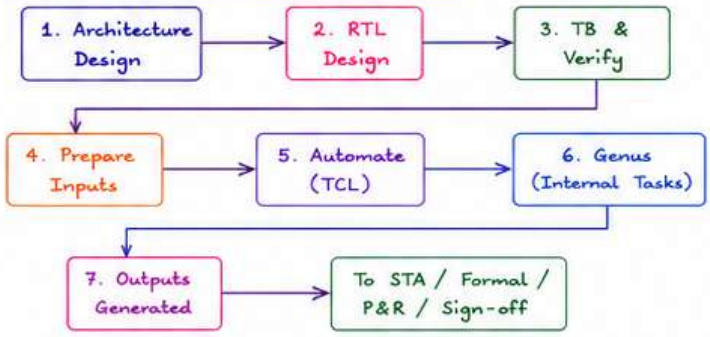
- Synthesized netlist : .v
- Timing constraints : .sdc (updated)
- Timing reports : .rpt
- Area reports : .rpt
- Power reports : .rpt
- DRC / Constraint reports
- Gate count / Cell utilization
- Synthesis log file : .log
- Power intent reports
- Design database (Milkyway / OA)

Output files
- → netlist.v
- → constraints.sdc
- → timing.rpt
- → area.rpt
- → power.rpt
- → drc.rpt / con.rpt
- → gate_count.rpt
- → synthesis.log
- → power_intent.rpt
- → design.db (MW/OA)

DETAILS OF KEY OUTPUT FILES :

File	Description	Usage
netlist.v	Synthesized gate level Verilog netlist	Input to STA, Formal Verification, P&R
constraints.sdc	Timing constraints (Clock, IO, Paths)	Used in STA, P&R and ECO
timing.rpt	Timing analysis report (setup/hold slack)	Check timing closure and violations
area.rpt	Area report (total, cell wise)	Analyze area and optimize
power.rpt	Power report (dynamic, leakage)	Power analysis and optimization
drc.rpt / con.rpt	Design rule / Constraint violations	Fix issues before sign-off
gate_count.rpt	Cell count and utilization	Check complexity and mapping
synthesis.log	Detailed synthesis log messages	Debug and trace errors / warnings
power_intent.rpt	UPF related checks & power structure	Verify power intent correctness
design.db	Milkyway / OA database	Used by P&R tools (Encounter, Innovus)

OVERALL SYNTHESIS FLOW IN CADENCE :



KEY POINTS :
- ✓ Proper inputs & constraints are critical for good results.
- ✓ Automation using TCL ensures consistency and saves time.
- ✓ Genus performs advanced optimizations for PPA.
- ✓ Outputs are used for downstream flows (STA, P&R, Sign-off).

INTERVIEW / VIVA QUESTIONS :

Q1. What is the role of Genus in Cadence synthesis flow ?
→ Genus is the synthesis engine responsible for optimization, mapping and generation of gate level netlist.

Q2. Why do we need SDC and UPF files in synthesis ?
→ SDC provides timing constraints and UPF provides power intent which help tool to optimize for timing and power.

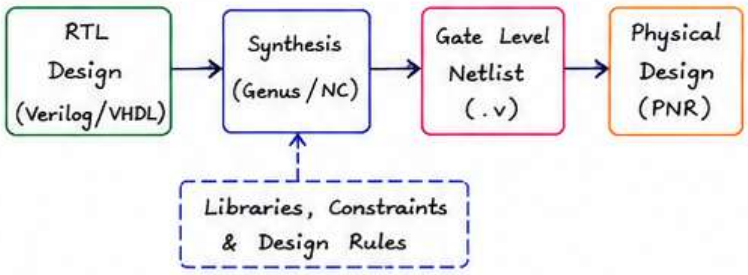
Q3. What are the main reports generated after synthesis ?
→ Timing, Area, Power, DRC, Constraint, Gate count reports etc.

PD SYNTHESIS — FROM RTL TO GATE LEVEL NETLIST

* INTRODUCTION :

PD Synthesis is the process of converting RTL (Register Transfer Level) description (Verilog / VHDL) into an optimized gate level netlist mapped to a target standard cell library, while meeting timing, area and power constraints.

* SYNTHESIS FLOW OVERVIEW :



* INPUTS TO SYNTHESIS :

1. RTL Code : Verilog / VHDL
2. Standard Cell Library : .db / .lib Files
3. Constraints (SDC) : Timing, clocks, I/O delays, False paths, Multicycle paths
4. Design Rules : Technology, Operating conditions
5. Other Files : UPF (Power intent), Test constraints (if any)

* LIBRARIES USED :

Library Type	Purpose
.db (Design Library)	Contains cell views, timing, area, power, etc.
.lib (Timing Library)	Contains timing, power, noise, constraint info.
.lef (Tech LEF)	Physical information of standard cells (not used in pure synthesis)
.tf (Tech File)	Technology specific rules

* STEPS - SIMULATION :

1. Source the Cadence environment
2. Launch NC launch in GUI : nclaunch -new
3. Add RTL Source file to the project
4. Run Multistep from the NC launch window
5. Open SimVision, add signals and verify waveforms

* STEPS - SYNTHESIS :

1. Launch Genus in legacy mode : genus -legacy_ui
2. Open the Genus GUI : gui_show
3. Source the run.tcl to map the design to tech libraries
4. This generates : .netlist, constraints, reports - Area, power, time, cells, gates
5. Review the gate schematic in Genus
6. (Optional) Save session and close

* MAIN STEPS IN SYNTHESIS :

1. **Design Read & Elaboration :**
   Reads RTL and builds the design hierarchy.
2. **Constraint Read & Check :**
   Reads SDC constraints and performs sanity check.
3. **Technology Mapping :**
   Maps RTL logic to standard cell library (creates initial gate level netlist).
4. **Logic Optimization :**
   Optimizes the design for timing, area, and power using various algorithms.
5. **Clock Tree Synthesis (Logical) :**
   Builds the clock tree logically (ideal clocks).
6. **Physical Awareness (Optional) :**
   Uses wireload / congestion / physical feedback for better estimation.
7. **Verification & Reporting :**
   Checks timing, area, power and design rule validity.

* OPTIMIZATION GOALS :

- ✓ Meet Timing Constraints (Setup, Hold)
- ✓ Minimize Area
- ✓ Minimize Power

} WITH MAXIMUM PERFORMANCE & RELIABILITY

* OUTPUTS FROM SYNTHESIS :

1. Gate Level Netlist : .v
2. Timing Constraints (post) : .sdc
3. Reports
   - Timing Report : .rpt
   - Area Report : .rpt
   - Power Report : .rpt
   - QoR Report : .rpt
4. Log File : .log
5. Design Database (optional) : .enc / .db

} Used as input to Physical Design (PNR)

* RTL TO GATE LEVEL NETLIST (EXAMPLE) :

RTL (Verilog)

```
module and2 (a,b,y);
 input a, b;
 output y;
 assign y = a & b;
endmodule
```

SYNTHESIS (Genus) →

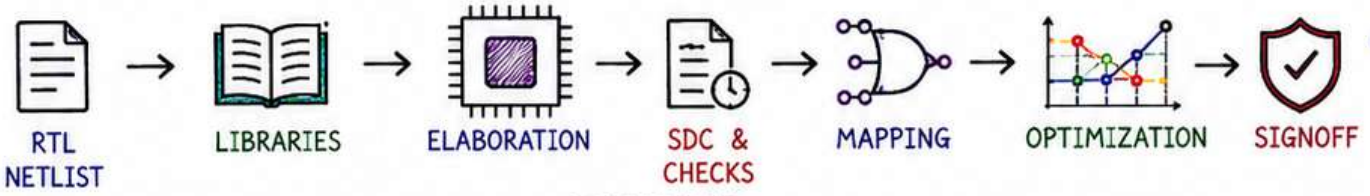
Gate Level Netlist (.v)

```
module and2 (a,b,y);
 input a, b;
 output y;
 U1 (.A(a), .B(b), .Y(y));
endmodule
// U1 -> AND2X1 cell
```

* KEY POINTS :

- Better constraints => Better QoR.
- Clean RTL + Good constraints => Accurate synthesis.
- Synthesis is the bridge between RTL design and Physical Design (PNR).
- Always verify timing, area and power before proceeding to PNR.

* Good Synthesis = Solid foundation for successful Physical Design.



PD SYN

THANK YOU!

Your time and attention are greatly appreciated.

IN

OUT

PD Synthesis – Transforming Design Intent into Optimized, Manufacturable, Signoff-Ready Layout.

PD SYNTHESIS OPTIMIZATION GOALS

- Timing Closure
- Area Minimization
- Power Efficiency
- Congestion Reduction
- Signal Integrity
- DFM Compliance

KEY ENGINES

TIMING ENGINE
- STA
- Slack Analysis
- Path Optimization

CONGESTION ENGINE
- Hotspot Detection
- Resource Balance
- Re-Routing

POWER ENGINE
- Power Estimation
- IR Drop Analysis
- Power Optimization

DRC / LVS ENGINE
- Rule Checking
- LVS Verification
- Clean Signoff

QOR ENGINE
- Area / Power / Timing
- Multi-Corner Analysis
- QoR Reporting

Thank you for being a part of this learning journey.
Keep Exploring, Keep Innovating, Keep Building the Future!

CLK

VDD

Author :
Konda Tharun Vivekananda